

Improvements Since First Deliverable

Project

Plane open-source project management platform

Purpose

This document explains what changed between the first deliverable change request reports and the current quality modernization work. The first deliverable consisted of three separate change request PDFs:

- `CR#1_Plane_Dospayev.pdf` : URL-based layout persistence
- `CR#2_Plane_Dospayev.pdf` : issue search debounce optimization
- `CR#3_Plane_Dospayev.pdf` : avatar initials edge-case fix

The current deliverable consolidates those change requests into a quality-focused modernization report with clearer metrics, stronger implementation choices, and updated verification evidence.

Executive Summary

The first deliverable identified three useful improvement areas and proposed localized fixes. The current version improves that work in four main ways:

- It reframes the work around software quality metrics instead of only individual bug reports.
- It strengthens the implementation by reducing unsafe TypeScript patterns.
- It aligns tests and documentation more closely with the repository's actual tooling.
- It records objective verification results from local test and type-check commands.

The current main report is `docs/quality-modernization-report.md` , with a generated PDF at `docs/quality-modernization-report.pdf` .

Overall Changes

Area	First deliverable	Current deliverable	Improvement
Structure	Three separate change request reports	One consolidated modernization report	Easier to map directly to the assignment prompt
Quality metrics	Mostly implicit through bug impact and tests	Explicit metrics: correctness, type safety, performance, regression coverage, build health	Stronger justification for assessment
Implementation maturity	Working patches with some shortcuts	Hardened code paths and clearer boundaries	Better maintainability
TypeScript quality	Some <code>any</code> and <code>@ts-nocheck</code> usage	Reduced unsafe casts and removed <code>@ts-nocheck</code> from the active project layout test	Better type safety
Verification	Test plans described in the PDFs	Commands were run and results recorded	More objective evidence
Version control	Patch-oriented	Suggested commit sequence by concern	Better version-control practice

CR#1: URL-Based Layout Persistence

First Deliverable

The first deliverable introduced URL-based layout persistence for project issue views. It described a feature where `?layout=kanban` could override the server-stored layout preference during project issue page initialization.

The report correctly identified the main quality problem: the application had two sources of truth for layout state, the server preference and the browser URL.

However, the first implementation used broad TypeScript escape hatches such as:

```
issuesFilter as any;
```

The first test file also used Jest-style globals and `@ts-nocheck`.

Current Improvements

- The current implementation keeps the same feature but improves code quality:
- `ProjectLayoutRoot` now uses the project-specific store interface, `IProjectIssuesFilter`.
 - It restores the existing typed store access pattern, `getIssueFilters(projectId)`.
 - It avoids broad `any` casts around `fetchFilters`, `updateFilters`, and `updateFilterExpression`.

- The active test file was converted from Jest-style globals to Vitest-style mocks.
- `@ts-nocheck` was removed from the project layout test.

Why This Is Better

The current version is more maintainable because it follows Plane's existing store patterns instead of bypassing them. It also gives a stronger quality argument for the assignment because the improvement is not only functional; it also improves type safety and reduces future regression risk.

Evidence

Touched files:

- `apps/web/core/components/issues/issue-layouts/roots/project-layout-root.tsx`
- `apps/web/core/components/issues/issue-layouts/roots/__tests__/project-layout-root.test.tsx`
- `apps/web/tests/cr1-url-layout-persistence.test.ts`

Verification:

- `pnpm.cmd --filter=web test` passed with 3 test files and 25 tests.

CR#2: Issue Search Debounce Optimization

First Deliverable

The first deliverable introduced a generic `useDebounce` hook and described its purpose: keep the search input responsive while delaying expensive downstream updates until the user pauses typing.

The original report framed the issue as render thrashing and potential network spam.

Current Improvements

The current work makes the implementation more accurate and repository-aware:

- The Space app now has a local hook at `apps/space/core/hooks/use-debounce.ts`.
- The Web debounce hook at `apps/web/core/hooks/use-debounce.ts` was normalized.
- A problematic default export before declaration was removed.
- `apps/space/core/components/issues/filters/selection.tsx` now passes the debounced query to child filter components.
- The report now describes the improvement more precisely as reducing avoidable child filter updates and render pressure.

Why This Is Better

The current version avoids overclaiming. Instead of saying the change definitely prevents backend spam in all cases, it focuses on what the code directly proves: the raw input state remains immediate, while expensive child filter components receive a delayed query value.

This is a stronger academic argument because the claimed metric matches the implemented behavior.

Evidence

Touched files:

- `apps/space/core/components/issues/filters/selection.tsx`
- `apps/space/core/hooks/use-debounce.ts`
- `apps/web/core/hooks/use-debounce.ts`

Verification:

- `pnpm.cmd --filter=space check:types` passed.
- `pnpm.cmd --filter=web test` passed.

CR#3: Avatar Initials Edge-Case Fix

First Deliverable

The first deliverable hardened the shared `getFirstCharacters` utility against edge cases:

- multiple spaces
- lowercase names
- empty strings
- null or undefined values

The original report kept the function signature as `str: string` while still defending against non-string runtime values.

Current Improvements

The current version makes the TypeScript contract match the runtime behavior:

```
export const getFirstCharacters = (str: unknown): string => {
```

The implementation is also simpler:

```
    return words
      .slice(0, 2)
      .map((word) => word.charAt(0))
      .join("")
      .toUpperCase();
```

The test file was updated to remove `@ts-nocheck` and use proper Vitest imports.

Why This Is Better

The current version is more honest and safer. If the function is designed to handle invalid values, then accepting `unknown` is clearer than pretending all callers pass a valid string.

The simplified implementation also reduces branching, making the function easier to read and maintain.

Evidence

Touched files:

- packages/utils/src/string.ts
- packages/utils/src/__tests__/string.test.ts
- packages/utils/tsconfig.json

Verification:

- pnpm.cmd --filter=@plane/utils check:types passed.

New Quality Metrics Added

The first deliverable explained impact per change request. The current deliverable makes the quality measurement explicit.

Metric	Added value
Functional correctness	Confirms behavior through regression tests
Type safety	Shows reduction of unsafe TypeScript patterns
Performance responsiveness	Justifies debounce as a responsiveness improvement
Regression coverage	Records tests that protect behavior
Build health	Uses test/type-check commands as objective evidence

Verification Summary

Command	Result	Meaning
pnpm.cmd --filter=web test	Passed	Existing web regression tests pass
pnpm.cmd --filter=@plane/utils check:types	Passed	Shared utility package type-checks
pnpm.cmd --filter=space check:types	Passed	Space app compiles with the new debounce hook
pnpm.cmd --filter=web check:types	Failed	Remaining failures are in pre-existing files outside this modernization pass

The web type-check failures are not from the current modernization work. They are located in:

- core/components/issues/filters.tsx
- core/components/views/form.tsx

What To Say In The Submission

Compared with the first deliverable, the current work is no longer just three separate bug-fix reports. It is now a structured quality modernization package. The selected components are justified, the quality metrics are explicit, the implementation has been hardened, and verification evidence has been recorded.

The most important improvements are:

- CR#1 now has stronger TypeScript store usage instead of broad `any` casts.
- CR#2 now has a repository-aware debounce implementation for Space and a cleaned Web hook.
- CR#3 now has a more accurate `unknown` input type and a simpler implementation.
- The final report includes quality metrics, verification results, limitations, and version-control guidance.

Remaining Limitations

- The web app still has unrelated type-check errors outside this work.
- The component-level `ProjectLayoutRoot` test is modernized but not included in the default Vitest run because the current test setup does not resolve `next/navigation` without additional aliasing.
- The local environment uses Node `22.17.0`, while the repo requests Node `>=22.18.0`, so pnpm shows an engine warning.